Extending the root filesystem (optional)

As mentioned in the requirements section, we're using a downloaded root filesystem file for the UML kernel.
If the compiled kernel modules you need to copy are many, or you need to copy other, large, files into the UML filesystem, then you probably need more space in the downloaded root filesystem. You can quickly resize the filesystem with the following three-step procedure:

1. Add space at the end of the filesystem file:

```
$ dd if=/dev/zero count=1024 bs=1024k >>
FedoraCore6-AMD64-root_fs
```

This adds 1 GB to the end of the root filesystem file. Be careful to use the '>>' (the double greater-than) redirection operator to append to it; if you use the single greater-than symbol, the rootfs will actually be an empty 1 GB file.

2. Do a forced check of the filesystem:

```
$ e2fsck -f FedoraCore6-AMD64-root_fs
```

3. Resize the filesystem to use the added space:

```
$ resize2fs FedoraCore6-AMD64-root_fs
```

First boot of UML

Now we are ready to boot UML for the first time.

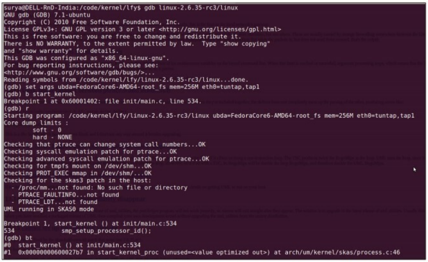
1) Boot a UML instance with the following command-line (shown in Figure 6):

```
$ linux-2.6.35.rc3/linux ubda=FedoraCore6-AMD64-
root_fs mem=256M
```

In this command-line, *ubda* specifies the filesystem image that is to be used as the root filesystem. If you need to pass more than one filesystem image, use arguments like *ubdb*, *ubdc*, etc. The optional *mem* parameter specifies the memory (RAM) that is allocated to the UML (it defaults to 128 M if not specified).

```
surya@DELL-RnD-India:/code/kernel/lfy$ ls
FedoraCore6-AMD64-root_fs  linux  linux-2.6.35-rc3  linux-2.6.35-rc3.tar.bz2  mods  root.fs
surya@DELL-RnD-India:/code/kernel/lfy$ linux-2.6.35-rc3/linux ubda=FedoraCore6-AMD64-root_fs mem=256M
Core dump limits :
        soft - 0
        hard - NONE
Checking that ptrace can change system call numbers...OK
Checking syscall emulation patch for ptrace...OK
Checking advanced syscall emulation patch for ptrace...OK
Checking for tmpfs mount on /dev/shm...OK
Checking PROT_EXEC mmap in /dev/shm/...OK
Checking for the skas3 patch in the host:
  - /proc/mm...not found: No such file or directory
  - PTRACE_FAULTINFO...not found
  - PTRACE_LDT...not found
UML running in SKAS0 mode
Adding 26873856 bytes to physical memory to account for exec-shield gap
```

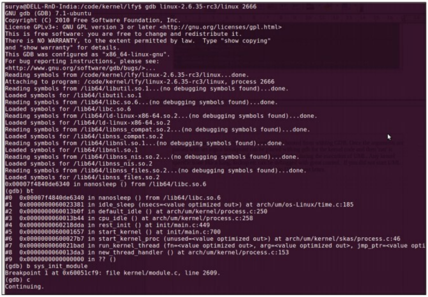
Figure 6: Booting UML



```
surya@DELL-RnD-India:/code/kernel/lfy$ gdb linux-2.6.35-rc3/linux
GNU gdb (GDB) 7.1-ubuntu
Copyright (C) 2010 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /code/kernel/lfy/linux-2.6.35-rc3/linux...done.
(gdb) set args ubda=FedoraCore6-AMD64-root_fs mem=256M eth0=tuntap,tap1
(gdb) b start_kernel
Breakpoint 1 at 0x60001402: file init/main.c, line 534.
(gdb) r
Starting program: /code/kernel/lfy/linux-2.6.35-rc3/linux ubda=FedoraCore6-AMD64-root_fs mem=256M eth0=tuntap,tap1
Core dump limits :
        soft - 0
        hard - NONE
Checking that ptrace can change system call numbers...OK
Checking syscall emulation patch for ptrace...OK
Checking advanced syscall emulation patch for ptrace...OK
Checking for tmpfs mount on /dev/shm...OK
Checking PROT_EXEC mmap in /dev/shm/...OK
Checking for the skas3 patch in the host:
  - /proc/mm...not found: No such file or directory
  - PTRACE_FAULTINFO...not found
  - PTRACE_LDT...not found
UML running in SKAS0 mode

Breakpoint 1, start_kernel () at init/main.c:534
534             smp_setup_processor_id();
(gdb) bt
#0  start_kernel () at init/main.c:534
#1  0x00000000600027b7 in start_kernel_proc (unused=<value optimized out>) at arch/um/kernel/skas/process.c:46
```

Figure 7: Debugging UML instance with GDB



```
surya@DELL-RnD-India:/code/kernel/lfy$ gdb linux-2.6.35-rc3/linux 2666
GNU gdb (GDB) 7.1-ubuntu
Copyright (C) 2010 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /code/kernel/lfy/linux-2.6.35-rc3/linux...done.
Attaching to program: /code/kernel/lfy/linux-2.6.35-rc3/linux, process 2666
Reading symbols from /lib64/libutil.so.1...(no debugging symbols found)...done.
Loaded symbols for /lib64/libutil.so.1
Reading symbols from /lib64/libc.so.6...(no debugging symbols found)...done.
Loaded symbols for /lib64/libc.so.6
Reading symbols from /lib64/ld-linux-x86-64.so.2...(no debugging symbols found)...done.
Loaded symbols for /lib64/ld-linux-x86-64.so.2
Reading symbols from /lib64/libnss_compat.so.2...(no debugging symbols found)...done.
Loaded symbols for /lib64/libnss_compat.so.2
Reading symbols from /lib64/libnsl.so.1...(no debugging symbols found)...done.
Loaded symbols for /lib64/libnsl.so.1
Reading symbols from /lib64/libnss_nis.so.2...(no debugging symbols found)...done.
Loaded symbols for /lib64/libnss_nis.so.2
Reading symbols from /lib64/libnss_files.so.2...(no debugging symbols found)...done.
Loaded symbols for /lib64/libnss_files.so.2
0x00007f4840de6340 in nanosleep () from /lib64/libc.so.6
(gdb) bt
#0  0x00007f4840de6340 in nanosleep () from /lib64/libc.so.6
#1  0x0000000060023381 in idle_sleep (nsecs=<value optimized out>) at arch/um/os-Linux/time.c:185
#2  0x0000000060013b0f in default_idle () at arch/um/kernel/process.c:250
#3  0x0000000060013b44 in cpu_idle () at arch/um/kernel/process.c:258
#4  0x0000000060218dda in rest_init () at init/main.c:449
#5  0x0000000060001657 in start_kernel () at init/main.c:700
#6  0x00000000600027b7 in start_kernel_proc (unused=<value optimized out>) at arch/um/kernel/skas/process.c:46
#7  0x000000006002lbad in run_kernel_thread (fn=<value optimized out>, arg=<value optimized out>, jmp_ptr=<value opti
#8  0x0000000060013da3 in new_thread_handler () at arch/um/kernel/process.c:153
#9  0x0000000000000000 in ?? ()
(gdb) b sys_init_module
Breakpoint 1 at 0x60051cf9: file kernel/module.c, line 2609.
(gdb) c
Continuing.
```

Figure 8: Attaching GDB to a running UML instance

2) Access the host filesystem in the UML instance, and copy the previously compiled kernel modules to UML's filesystem. This can be done in various ways; I am highlighting a couple of methods here.

a. Hostfs method (root access not required): Hostfs is a UML filesystem that